

From physical quantity to trustworthy samples

Lecture 3 of 16 · 80 minutes · Module A: Foundations

One log file, and three numbers that decide how much of it is true.

Last week's muddiest points

Your questions, answered before we start

1

INSTRUCTOR: fill these three in from Lecture 2's exit tickets before class.

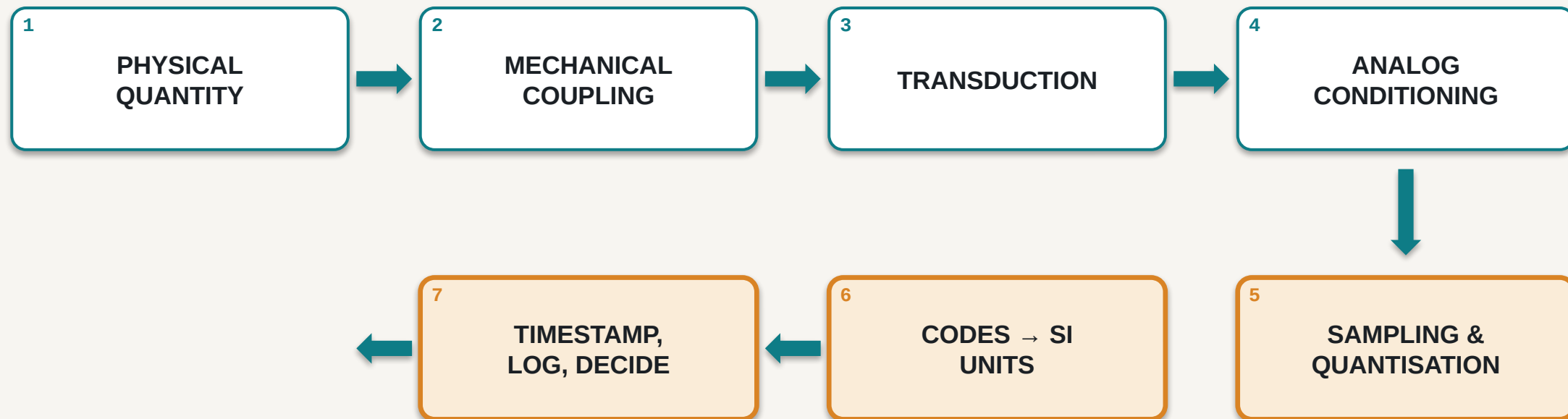
2

3

Refer to concepts, never to students. Ninety seconds total, then move on.

Where are we today?

Retrieval: the chain from Lecture 1 — today we live in boxes 5, 6 and 7



Stages 1–4 hand you a conditioned signal. Everything after that is arithmetic on numbers — and three of its errors cannot be undone.

One log file

Ten minutes of data. 60 000 rows. No gaps, no outliers, plausible magnitudes.

ISM330DHCX · ± 2 g · 16-bit · ODR = 104 Hz · 0.061 mg/LSB · HAL_Delay(10)
polling loop · 10 minutes

12.4

**BITS OF SIXTEEN THAT CARRY
INFORMATION**

You bought a 16-bit number.
You own rather less than that.

12 Hz

**A SIGNAL IN YOUR DATA THAT DOES NOT
EXIST IN THE WORLD**

It is not noise. It is a tone,
and it looks entirely real.

12.1 s

**HOW WRONG YOUR TIMESTAMPS ARE BY
THE END**

Every event in the last part of
the log is stamped too early.

**Every one of these three was computable before a line of firmware was written.
Two of the three cannot be repaired afterwards at any price.**

POLL 1

Your accelerometer is 16-bit, configured for ± 2 g, so its datasheet resolution is 0.061 mg/LSB. You configure ODR = 104 Hz and log the output. What is the smallest change in acceleration your log can actually distinguish?

- A** 0.061 mg — that is what the datasheet says
- B** 0.018 mg — the quantisation noise, $\text{LSB}/\sqrt{12}$
- C** 0.72 mg
- D** It cannot be determined from what you have been told

Commit to one. The answer is withheld until the end of the lecture — and by then you will have computed it yourselves.

The number everyone trusts

Where 0.061 mg/LSB comes from

$$4000 \text{ mg} \div 65\,536 \text{ codes} = 0.061 \text{ mg / LSB}$$

One count of the register is 0.061 mg. That much is simply true.

± 2 g is 4000 mg of span. Sixteen bits is 65 536 codes. Divide one by the other and you get the size of one step of the number — which is exactly what the datasheet prints, on the same page as everything else we will use today.

...and three things it is not

It is not the smallest change you can distinguish.

That is set by the noise — and the noise depends on a bandwidth nobody has chosen yet.

It is not an accuracy.

Offset, scale error and drift are separate numbers on separate rows. Lecture 2 spent an hour on them.

It is not a property of the sensor alone.

Change the full scale and it moves. Change the bandwidth and the measurement changes while it stays put.

**ON THE BOARD, NOW, AND NOT ERASED:
you bought 16 bits — how many of them do you own?**

The output and the reference

What actually comes out of a sensor — and what the converter compares it against.

One wire's difference between a measurement that survives a sagging rail and one that does not.

What comes out of a sensor

Three analog forms — and one where the conversion already happened, where you cannot see it

VOLTAGE

Tens of μV per $^{\circ}\text{C}$ from a thermocouple; a few mV from a bridge.

Small enough that the wire is part of the measurement.

CURRENT · 4–20 mA

Unchanged by cable resistance, so distance does not matter.

4 mA is a live zero: a broken wire reads 0 mA, and is detectable.

BRIDGE

Four elements in a diamond, excited by a voltage; the output follows the imbalance. Strain gauges, pressure sensors, load cells.

DIGITAL · I²C / SPI

The ADC is inside the package. You receive a number, not a voltage. Almost every MEMS sensor here — and today's case.

VOCABULARY ONLY TODAY: instrumentation amplifier · common-mode range · grounding and shielding →
Lecture 12 in full

A digital output does not remove the analog problems — it moves them inside the package, where you can neither see nor change them.

And it adds two of its own: the byte order of the result, and when the conversion happened.

What an ADC actually measures

Not a voltage — a ratio

$$\text{count} = V_{\text{in}} / V_{\text{ref}} \times 2^N$$

An ADC has no idea what a volt is.

► **It is a ratio, not a voltage.**

The count depends on the input compared with the reference. Read the formula again for what it says.

► **Every V_{ref} error is a gain error.**

Not on one reading — on every reading, in the same direction, for as long as the reference is wrong.

The rule: excite the sensor and reference the converter from the same thing, or from two things that are both stable.

Two wirings, one sagging rail

A 3.3 V rail sags to 3.2 V under load. Representative bridge figures, not a named product.

RATIOMETRIC — bridge and ADC reference from the SAME rail

bridge output ↓ 3 %

ADC counts ↑ 3 %

reported strain 0 % error

The code computes a ratio, and both terms moved together, so the ratio never changed. This is free — if you wired it deliberately.

ABSOLUTE — precision 2.5 V reference, bridge still on the rail

bridge output ↓ 3 %

ADC counts unchanged

reported strain 3 % LOW

Nothing cancels. And it is not a constant — it varies with whatever else the board is doing, so no calibration removes it.

Same components. Same cost. One wire's difference.

A strain-gauge bridge is excited from the microcontroller's 3.3 V rail. The ADC that reads it uses the same 3.3 V rail as its reference. Under load, the rail sags to 3.2 V. The reported strain:

- A** reads about 3 % high
- B** reads about 3 % low
- C** does not change
- D** changes by an amount that depends on the gauge factor

Vote alone first. Then find someone who voted differently and make them defend it.

A strain-gauge bridge is excited from the microcontroller's 3.3 V rail. The ADC that reads it uses the same 3.3 V rail as its reference. Under load, the rail sags to 3.2 V. The reported strain:

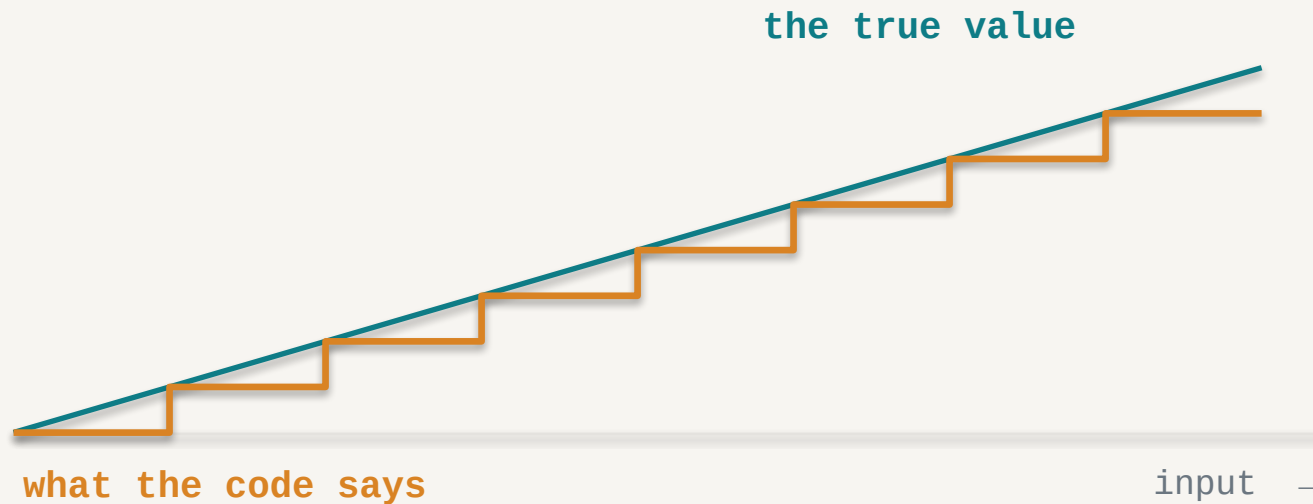
- A reads about 3 % high
- B reads about 3 % low
- C does not change**
- D changes by an amount that depends on the gauge factor

The bridge output is proportional to its excitation; the counts are proportional to $1/\text{reference}$. Both scale with the same rail, so the ratio the code computes never moved.

Quantisation

The converter has a finite number of codes, so it must round

$$\text{LSB} = \text{full scale} / 2^N = 4000 \text{ mg} / 65\,536 = \mathbf{0.061 \text{ mg}}$$



Between two codes there is nothing.

The converter reports the nearest step and throws the difference away. The error is uniformly distributed over one LSB, and it is there in every single sample.

It is also the only error in this lecture that is perfectly predictable before you switch anything on.

The quantisation noise

Rounding injects a known, computable amount of it

$$\sigma_q = \text{LSB} / \sqrt{12} = 0.061 / 3.464 = \mathbf{0.018 \text{ mg}}$$

$\sqrt{12}$ is the standard deviation of a uniform distribution of unit width. Worth knowing where it comes from once; not worth deriving twice.

Note what this is NOT: it is the resolution of the number, not the resolution of the measurement.

**Hold on to 0.018 mg.
Later we put it beside the sensor's own noise — and one of those two terms decides nothing at all.**

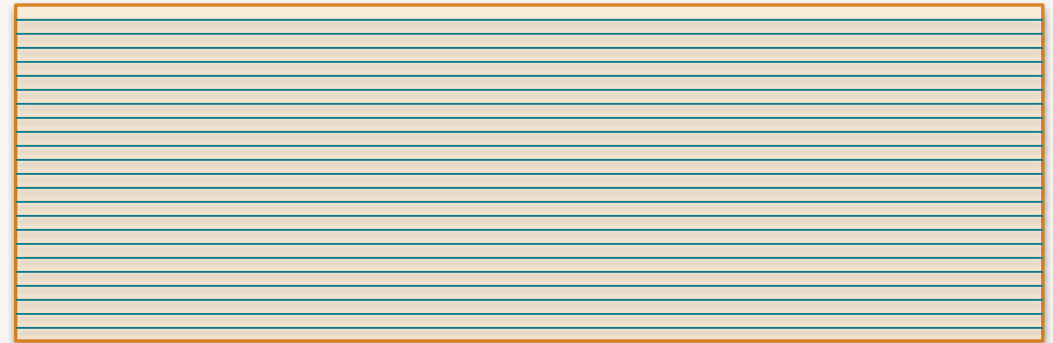
Why more bits is not more truth

the noise floor of the measurement — identical in both



16-BIT CONVERTER

one step = 0.061 mg



20-BIT CONVERTER

one step = sixteen times finer

**Adding bits divides the step. It does not divide the noise.
Bits below the noise floor are extra digits, not extra information.**

Whether the last bits buy you anything depends on a number that is not on the front page — the noise density — and on a choice that is entirely yours: the bandwidth.

Draw the last three boxes of the chain from memory

Under each one, write the single thing that can be lost there.

SAMPLING

→

CODES TO UNITS

→

TIMESTAMP

Turn your paper over first. Nothing to hand in.

Output data rate is not bandwidth

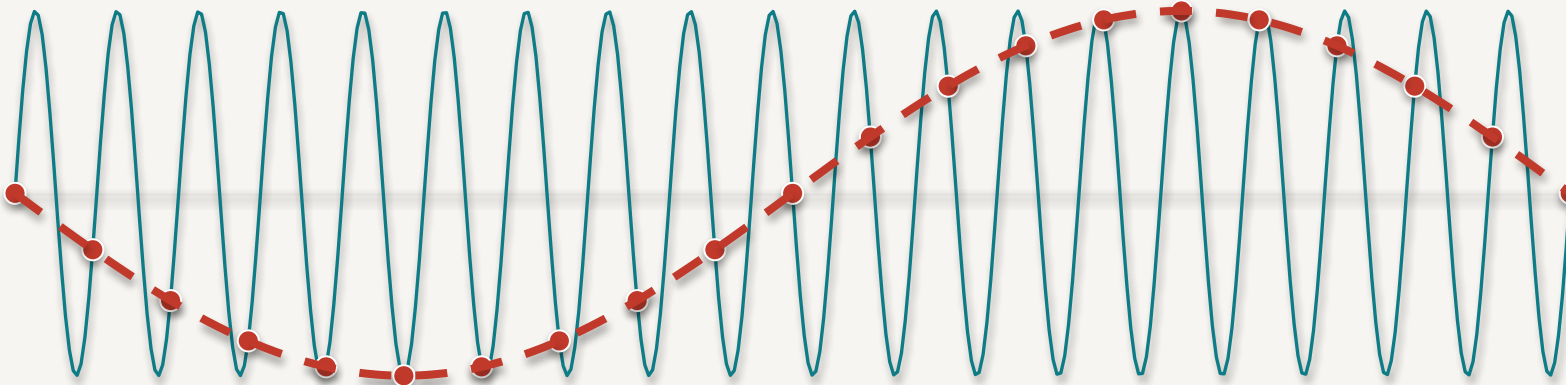
The hardest idea in Module A.

One register bit decides whether your data contains a signal that never existed.

Nyquist, as a design rule

Lecture 1 met this as a discovery. Today it is a register setting.

the real signal · 19 cycles



what the samples reconstruct · 1 cycle

$$f_s > 2 \times f_{\max}$$

The theorem is not the design rule.

The design rule is its contrapositive: you must guarantee that nothing above $f_{\max}$ ever reaches the sampler.

The samples are entirely correct. Their interpretation is not.

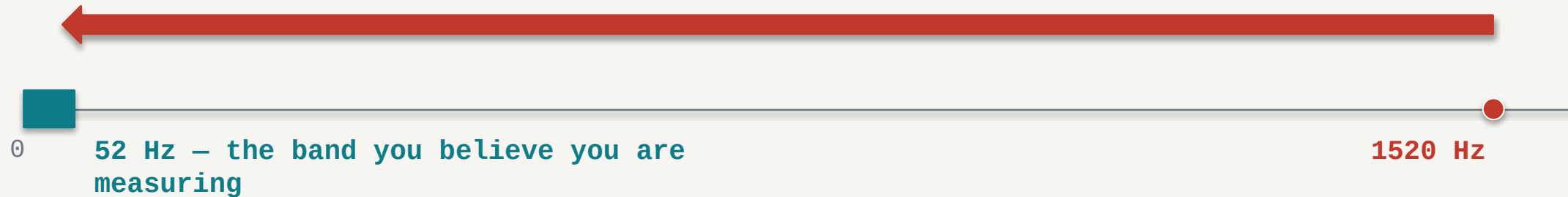
Where a tone lands after sampling

N is whichever integer multiple of the sample rate lies nearest to it

$$f_{\text{alias}} = |f - N \times f_s|$$

$$\text{Lecture 1's bearing tone, at today's ODR: } |1520 - 15 \times 104| = |1520 - 1560| = 40 \text{ Hz}$$

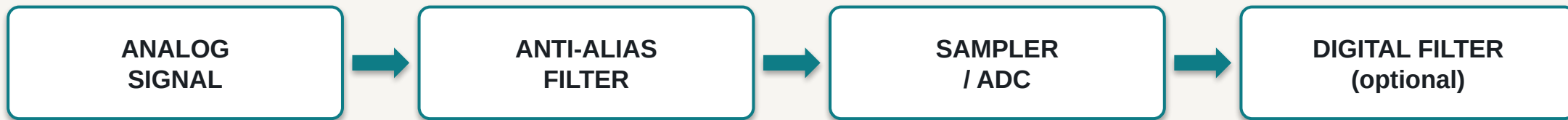
folds back into the band you are keeping



The sample rate alone tells you nothing about what is in your data.

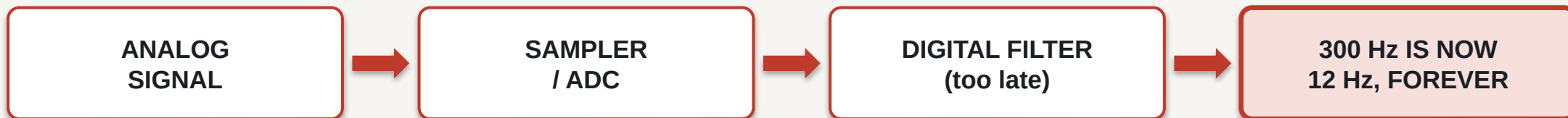
The one ordering that cannot be repaired

CORRECT — the filter sits before the sampler



Out-of-band energy is removed while it is still separable from the signal. Everything downstream is then a choice, not a repair.

WRONG — “we will filter it in software afterwards”



By the time the data exists, a 300 Hz component and a genuine 12 Hz component are the same sequence of numbers. Not similar — identical. Nothing separates two things that are the same.

Everything else in the chain — a wrong gain, a missed sign, a unit muddle — can be fixed on a laptop a week later. This one cannot.

The bit that decides it

ISM330DHCX · CTRL1_XL (0x10) · reset value 0x00



LPF2_XL_EN = 0 after reset. Your part ships with no anti-alias filter.

So the natural sequence of events is this. You set ODR = 104 Hz. You reason, correctly, that the Nyquist limit is 52 Hz. You write “52 Hz measurement bandwidth” in the report.

Meanwhile the device is sampling a far wider analog band with no filter at all, and everything above 52 Hz folds back into your data.

0x40 gives ±2 g at 104 Hz — and leaves bit 1 clear. In Laboratory 2 you set it yourself.

You configure the accelerometer for ODR = 104 Hz and leave CTRL1_XL bit LPF2_XL_EN at its reset value of 0. A pump on the same frame vibrates at 300 Hz. What appears in your logged data?

- A** nothing — 300 Hz is above the sample rate, so it is not sampled
- B** a 300 Hz component, attenuated
- C** a 12 Hz component, indistinguishable from real signal
- D** broadband noise, raising the noise floor slightly

Vote. Then two minutes in pairs: find someone who voted differently and make them defend it. Then we vote again — and only then do we do the arithmetic.

You configure the accelerometer for ODR = 104 Hz and leave CTRL1_XL bit LPF2_XL_EN at its reset value of 0. A pump on the same frame vibrates at 300 Hz. What appears in your logged data?

- A nothing — 300 Hz is above the sample rate, so it is not sampled
- B a 300 Hz component, attenuated
- C a 12 Hz component, indistinguishable from real signal**
- D broadband noise, raising the noise floor slightly

$|300 - 3 \times 104| = |300 - 312| = 12 \text{ Hz}$. Squarely inside the band a tilt or vibration measurement cares about, slow enough to look like real mechanical behaviour, and permanent.

Three numbers, three jobs

The sentence most often got wrong on the midterm

	What it decides	Who sets it	On your part, today
ODR — output data rate	how often a new number appears at the output	you, by register	104 Hz · CTRL1_XL bits 7:4
Measurement bandwidth	the highest frequency reported faithfully	you — often by a different register	52 Hz · only if LPF2 is on
Anti-alias cut-off	what reaches the sampler at all	the analog path — sometimes nobody	none · LPF2_XL_EN = 0

ODR ≠ BW ≠ f_cut

Set the ODR wrongly and you get more numbers, or fewer, and you waste bus and power.

Set the cut-off wrongly and the data is wrong for ever.

Sampling faster does not fix it

Same pump, three configurations

Interfering tone	Sampled at	Appears at	
300 Hz — a pump on the same frame	104 Hz	12 Hz	inside your band
1520 Hz — Lecture 1's bearing	104 Hz	40 Hz	inside your band
300 Hz — the same pump	100 Hz	0 Hz	it looks like an offset

At 100 Hz the pump becomes a DC offset. Somebody will spend a productive week calibrating it out — and will succeed, at that one pump speed. Succeeding is worse than failing: the calibration is then wrong at every other speed, and the sensor gets the blame.

Raising the sample rate moves the alias. It does not remove it. Choose the band first, then the rate.

Codes to trustworthy values

Four conversions between the die and the CSV file.

Each one has a way of going wrong that produces a perfectly plausible answer.

One register read, all the way to SI

Ask the class for each step before you write it

OUTX_L_A (0x28) = 0x2C OUTX_H_A (0x29) = 0xFF

↓ *little-endian: the low byte is at the lower address*

raw = 0xFF2C = 65 324 as unsigned

↓ *16-bit two's complement: bit 15 is set, so it is negative*

65 324 - 65 536 = -212 counts

↓ *× sensitivity, ±2 g → 0.061 mg/LSB*

-212 × 0.061 mg = -12.9 mg

↓ *→ SI, × 9.80665*

-0.0129 g × 9.80665 = -0.127 m/s²

Four steps.

Four ways to get an answer that looks entirely reasonable.

All four turn up in real laboratory submissions.

The device is very nearly at rest, tilted slightly, on this axis.

Four ways to a plausible wrong answer

All four appear in real laboratory submissions

The mistake	What you get	Why it is dangerous
Byte order — high byte read first	0x2CFF = 11 519 = +703 mg	a wrong answer of a believable magnitude, which is the dangerous kind
Sign ignored — word read as unsigned	65 324 = +3985 mg	a device at rest reporting nearly 4 g; at least this one announces itself
IF_INC — CTRL3_C written 0x00 “to start clean”	six copies of one byte	CTRL3_C resets to 0x04, so auto-increment was already on. You turned it off
Units left implicit	mg, g and m/s ² in one file	put the unit in the column header, once, and never in the prose

The plausibility check that tests the whole chain at once: at rest, one axis reads about 9.81 m/s², and the vector sum of all three is 1 g in any orientation.

The bits you own

Every figure below is already in the datasheet you have open

Quantity	Arithmetic	Result
full scale, ± 2 g	—	4000 mg
codes, 16-bit	2^{16}	65 536
LSB	$4000 / 65\ 536$	0.061 mg
quantisation noise	$0.061 / \sqrt{12}$	0.018 mg
bandwidth, ODR 104 Hz, LPF2 on	$104 / 2$	52 Hz
sensor noise, 100 $\mu\text{g}/\sqrt{\text{Hz}}$ (max)	$100 \times \sqrt{52}$	0.721 mg
noise, in LSB	$0.721 / 0.061$	11.8 LSB
bits that are noise	$\log_2 11.8$	3.6 bits
EFFECTIVE BITS	$16 - 3.6$	12.4 bits

The quantisation noise is forty times smaller than the sensor noise. Every argument about the last bit of an ADC is, in this system, an argument about nothing.

You bought 16 bits. You own 12.4.

With the datasheet's typ noise density of $60 \mu\text{g}/\sqrt{\text{Hz}}$ the same arithmetic gives 0.433 mg and 13.2 effective bits. The part's own two printed columns move the answer by 0.8 bits — Lecture 2's lesson, restated in a new quantity.

The clock you did not have

HAL_Delay(10) is not a 100 Hz sample rate

I²C read · 202 µs

HAL_Delay(10) · 10.000 ms

one loop iteration — drawn to scale

six bytes over I²C at 400 kHz
81 bit-times ÷ 400 kHz = 202 µs

10.000 + 0.202 = 10.203 ms
→ 98.0 Hz, not 100 Hz

Two per cent. Always in the same direction. Every single iteration.

Nothing is broken. Nothing reports an error. The loop simply takes longer than you told it to
—
and a delay is a minimum, never a period.

What 203 microseconds does to ten minutes

	You believe	Reality
Sample interval	10.000 ms	10.203 ms
Sample rate	100 Hz	98.0 Hz
A real 20 Hz tone is reported at	20 Hz	20.41 Hz
After 60 000 samples the log says	600.0 s	612.1 s
Every event is stamped	when it happened	12.1 s too early

**Worse than a constant scaling error: the loop period depends on which branches ran, whether an interrupt arrived, and what the bus was doing.
You can bound this error. You cannot invert it.**

The fix, which is free

Let the sensor decide when the sample happened

	Polling with a delay	Data-ready interrupt
Sets the interval	your software	the sensor's own timebase
Interval error	loop-dependent, unbounded	specified in the datasheet
Cumulative time error	grows without limit	bounded by the timebase tolerance
CPU cost	high — busy waiting	low
Effort to implement	slightly less	slightly more

The sensor knows when it sampled. Ask it, rather than guessing.

Timestamp from a hardware timer, not a loop counter: the error becomes a published tolerance instead of your software's mood. Laboratory 2 asks for a timing plot for exactly this reason.

Knowing that the data is real

Four cheap checks, each catching a failure that otherwise looks like bad data

WHO_AM_I (0x0F)

Read it and compare it with 0x6B.

One transaction proves the address, the bus, the pull-ups and the supply, all at once.

SELF-TEST

Deflect the proof mass electrostatically by a known amount.

The only check here that tests the mechanics rather than the electronics.

FIFO OVERRUN

If the buffer overflowed, samples were lost and the rest are not evenly spaced.

A dropped sample is a timing error disguised as data.

BUS ERROR & RECOVERY

I²C can hang with a slave holding the data line low.

A design that cannot recover stops logging one day and gives no reason.

All four are omitted from most first attempts. All four turn “the data looks strange” into “the sensor was never responding”.

Your accelerometer is 16-bit, configured for ± 2 g, so its datasheet resolution is 0.061 mg/LSB. You configure ODR = 104 Hz and log the output. What is the smallest change in acceleration your log can actually distinguish?

- A 0.061 mg — that is what the datasheet says
- B 0.018 mg — the quantisation noise, $\text{LSB}/\sqrt{12}$
- C 0.72 mg**
- D It cannot be determined from what you have been told

0.72 mg — nearly twelve times the LSB. Your first answer should have been D: you had not been given the noise density. The right answer now is C, and you computed it yourselves.

A water-level logger reads a 4–20 mA pressure loop through a 100 Ω sense resistor and a 12-bit ADC, polling in a `while()` loop with a 50 ms delay. It runs for a week. Afterwards you discover all four of the following. Which one can you still fix?

- A** the reported level is 40 mm high across the whole week
- B** the timestamps drift, ending 90 s behind real time
- C** a 9.7 Hz component from the pump has folded into the band
- D** the last two ADC bits were always noise

Different measurand, different interface, different failure — the same question underneath. Vote, then hold your answer: the next slide is the answer to this one.

Calibratable, or gone

The table to keep — and Poll 4's answer is its first row

Error	Removable afterwards?	How, or why not
Offset	YES	one-point calibration against a known reference
Scale factor	YES	two-point calibration
Byte order, sign, units	YES	reinterpret the file — the bits are all there
Temperature drift	PARTLY	only if you logged the temperature. So log it
Random noise	PARTLY	averaging reduces it by $\sqrt{N}$, and costs you bandwidth
Quantisation	NO — but negligible here	0.018 mg against 0.721 mg of noise
Aliasing	NO	the alias and the signal are the same numbers
A lost or wrong timestamp	NO	the timing information was never recorded

**Everything above is a transformation of data that is still there. Everything below is missing information.
No amount of processing creates information.**

Three numbers, one log file

All three were computable before a line of firmware was written

12.4

BITS THAT CARRY INFORMATION

$16 - \log_2(0.721 / 0.061)$, from the noise density and your bandwidth.

12 Hz

A SIGNAL THAT DOES NOT EXIST IN THE WORLD

$| 300 - 3 \times 10^4 |$. One register bit, left at its reset value of 0.

12.1 s

OF TIMESTAMP ERROR AFTER TEN MINUTES

$60\,000 \times 0.203$ ms. One I²C read the loop never accounted for.

**One of the three you can state and live with. Two of them cannot be repaired afterwards at any price —
and neither of those two announces itself anywhere in the file.**

The same three numbers you were shown at the start. You now know where every one of them came from.

The four claims a trustworthy sample makes

01 What is it?

A value in SI units — with the sensitivity, the sign convention and the byte order you assumed, stated.

02 How much of it is information?

The effective resolution, from the noise density and YOUR bandwidth. 12.4 bits, not 16.

03 Which band does it represent?

The measurement bandwidth, and evidence that a filter sat below it, before the sampler. Not the ODR.

04 When did it happen?

A timestamp whose error you can state, from a source you can name.

A log file that cannot answer these four is not data. It is a plausible file.

Week 4: Laboratory 2

Sampling, aliasing and the ADC — with the register bits in your hands

PRE-LAB, before you arrive: compute the effective bits for TWO ODR settings, exactly as we did today.

The bench time is for measuring, not for deriving — and you will be asked for a timing plot.

Bring one answer with you: which bit do you set so that 52 Hz actually means 52 Hz?

AND THEN LECTURE 4

MEMS structures, transduction, fabrication and packaging — why the numbers in the datasheet are the numbers they are, and where drift comes from before anybody switches anything on.

Exit ticket

Ninety seconds · on paper · handed in at the door

1 · JUDGEMENT

A colleague says:

“we will sample at 1 kHz and filter it down in software afterwards.”

In one sentence, what is wrong with that plan?

2 · RETRIEVAL

Name one error in your Laboratory 1 data that you now believe was IRREVERSIBLE.

One line. No explanation needed.

Question 2 is the one that matters. It is the input to Lecture 4's opening slide, so it will actually be read.